

Grado en Ingeniería Informática  
Fundamentos de Programación II 25/26

**Hito 4:**  
**Proyecto práctico: Convocatoria  
extraordinaria**

## Resumen

En esta convocatoria extraordinaria se plantea una recuperación del proyecto práctico desarrollado durante el curso. El trabajo consiste en revisar el sistema construido en los hitos anteriores, corregir los problemas que hayan impedido alcanzar los objetivos mínimos y cerrar una versión integrada del videojuego RPG.

Además, se incorporará una ampliación final relacionada con el peso de los objetos y con el movimiento de objetos entre personajes y tienda. Esta ampliación tiene un alcance reducido, pero obliga a razonar sobre el estado del sistema cuando una operación afecta a más de una parte del juego.

**IMPORTANTE:** La entrega deberá funcionar como un proyecto completo. La ampliación del Hito 4 se evaluará sobre una versión corregida e integrada de los hitos anteriores.

## 1. Objetivos

- Revisar, corregir e integrar el trabajo realizado en los hitos anteriores.
- Mantener la coherencia entre análisis, diseño e implementación en una versión completa del proyecto.
- Ampliar el sistema desarrollado teniendo en cuenta el peso de los objetos.
- Gestionar operaciones que mueven objetos entre distintas partes del sistema sin dejar estados inconsistentes.
- Consolidar el proceso de trabajo por fases: análisis, diseño e implementación.

## 2. Normas de trabajo

El ejercicio se entregará en la tarea habilitada en Campus Virtual para la convocatoria extraordinaria. Salvo indicación expresa del profesorado, se deberán seguir estas normas:

- **Entrega**
  - Todos los archivos deberán comprimirse en un único archivo ZIP.
  - El nombre del ZIP debe seguir el formato `LX_XX_H4_EXTRA.zip` según el grupo de Campus Virtual.
  - El ZIP deberá incluir:
    - Imagen del diagrama de clases y relaciones UML (`.png` o `.jpg`).
    - Únicamente la carpeta `src` con los archivos `.java`.
- **Restricciones de implementación**
  - No utilizar múltiples `return` en una misma función.

- No utilizar `continue`.
- No utilizar `break`, salvo en sentencias `switch`.
- No utilizar `ArrayList`.
- Queda totalmente **prohibido el uso de Inteligencia Artificial** para realizar estas prácticas. Cualquier indicio de su utilización será penalizado.

### 3. Fases del desarrollo orientado a objetos

Se pide seguir las fases del desarrollo de software orientado a objetos:

- **Análisis:** En esta fase se identifican los requisitos del sistema. Intentamos encontrar los objetos y relaciones adecuados para el problema propuesto. Investigamos el problema centrándonos en los conceptos del dominio del problema, sin preocuparnos por la implementación. Es importante entender el problema y definir claramente los requisitos antes de pasar a la siguiente fase.
- **Diseño:** En esta fase se define la arquitectura del sistema. Partiendo de los objetos y relaciones identificados en el análisis, se diseña la estructura del sistema. Se definen las clases, sus atributos y métodos (aún sin pensar en la implementación, solo en aquella funcionalidad que cada clase debe tener), así como las relaciones entre ellas (herencia, composición, etc.). Es importante diseñar un sistema modular y flexible que permita cambios futuros sin afectar a toda la aplicación. En esta fase es fundamental el uso de diagramas UML (especialmente el diagrama de clases).
- **Implementación:** Únicamente cuando tenemos claro el diseño, pasamos a la implementación. En esta fase se escribe el código fuente de la aplicación siguiendo el diseño establecido. Es importante seguir buenas prácticas de programación y mantener el código limpio y organizado.

### 4. Desarrollo de la práctica

En esta recuperación se parte del sistema desarrollado en los hitos anteriores. Antes de incorporar la ampliación final, el proyecto deberá revisarse para que mantenga de forma integrada la funcionalidad trabajada durante el curso: lectura de datos desde ficheros, gestión de personajes, inventarios, equipo, tienda, compra de objetos, tratamiento de errores y escritura de información del sistema cuando corresponda.

El programa debe arrancar correctamente, permitir la navegación por el menú y conservar la coherencia entre los datos leídos, los objetos creados durante la ejecución y la información que se muestra al usuario. Si al recuperar una parte del proyecto se rompe otra funcionalidad previa, se considerará un problema de integración.

A partir de esa base, el sistema deberá tener en cuenta que los objetos ocupan peso. En los hitos anteriores el peso formaba parte de la información de cada objeto, pero no limitaba las acciones del personaje. En esta versión, cada personaje podrá cargar como máximo **20 unidades de peso**, de modo que no todos los movimientos de objetos serán válidos.

Para representar esta nueva situación anómala, se deberá incorporar la siguiente excepción propia:

- **PesoMaximoSuperadoException:** se producirá cuando se intente añadir o mover un objeto al inventario de un personaje y con ello se supere el peso máximo permitido.

La ampliación se centra en operaciones en las que un objeto sale de un lugar del sistema y

pasa a otro. Además de la compra ya incorporada en hitos anteriores, se añadirá la posibilidad de que un personaje entregue un objeto a la tienda a cambio de dinero y de que un objeto pase de un personaje a otro. Estas operaciones deberán integrarse en el menú de forma natural, manteniendo el estilo de interacción usado hasta ahora.

El punto importante de este hito es la consistencia del estado. Si una operación no puede completarse porque falta espacio, porque se supera el peso permitido o porque la posición seleccionada no contiene ningún objeto, el programa deberá tratar la situación como un error del dominio y el sistema deberá quedar como estaba antes de intentar la acción. No debe perderse ni duplicarse ningún objeto, y el dinero de los personajes solo deberá cambiar cuando la operación correspondiente haya terminado correctamente.

El tratamiento de estas nuevas situaciones deberá seguir la línea del Hito 3, incorporando las excepciones propias que sean necesarias y capturándolas en el nivel adecuado del programa. También se deberá actualizar el diagrama UML para reflejar la solución diseñada y explicar en la memoria final (en caso de que se desee recuperar también esta parte) qué problemas se han corregido de los hitos anteriores y cómo se ha integrado la ampliación del Hito 4.

#### 4.1. Ficheros de entrada

No se modifica el formato de los ficheros `personajes.txt` y `tienda.txt`. La ampliación deberá trabajar con la información ya disponible en esos ficheros, manteniendo la compatibilidad con la estructura utilizada en los hitos anteriores.